

Suite TD

```
Dlist *dlist_remove_all(Dlist *p_list, int data)
{
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_head;
        while (p_temp != NULL)
        {
            if (p_temp->data == data)
            {
                struct node *p_del = p_temp;
                p_temp = p_temp->p_next;
                if (p_del->p_next == NULL)
                {
                    p_list->p_tail = p_del->p_prev;
                    p_list->p_tail->p_next = NULL;
                }
                else if (p_del->p_prev == NULL)
                {
                    p_list->p_head = p_del->p_next;
                    p_list->p_head->p_prev = NULL;
                }
                else
                {
                    p_del->p_next->p_prev = p_del->p_prev;
                    p_del->p_prev->p_next = p_del->p_next;
                }
                free(p_del);
                p_list->length--;
            }
            else
            {
                p_temp = p_temp->p_next;
            }
        }
    }
    return p_list;
}
```

9. Ecrire une fonction de recherche. Cette fonction de recherche sera un tout petit peu particulière. En effet, celle-ci ne renverra pas l'élément qu'elle aura trouvé mais une liste contenant l'élément qu'elle aura trouvé. Ceci facilitera de ce fait notre gestion. Comment faire ? Vous vous souvenez de la fonction de suppression, Et bien nous allons utiliser le même style. Nous allons parcourir notre liste tant que nous n'aurons pas trouvé notre élément (variable found). Si nous trouvons notre élément, nous utilisons alors les fonctions déjà à notre disposition (dlist_new et dlist_append) pour créer notre liste qui sera retourné.

```
Dlist *dlist_find(Dlist *p_list, int data)
{
```

```
Dlist *ret = NULL;
if (p_list != NULL)
{
    struct node *p_temp = p_list->p_head;
    int found = 0;
    while (p_temp != NULL && !found)
    {
        if (p_temp->data == data)
        {
            ret = dlist_new();
            ret = dlist_append(ret, data);
            found = 1;
        }
        else
        {
            p_temp = p_temp->p_next;
        }
    }
}
return ret;
}
```

10. Pour compléter notre fonction de recherche, nous allons maintenant créer une fonction qui non pas s'arrête au premier élément trouvé, mais qui retourne tous les éléments trouvés, il s'agit exactement du même code que précédemment, mais cette fois ci, sans l'utilisation de la variable found. Ainsi, toute la liste est parcourue :

```
Dlist *dlist_find_all(Dlist *p_list, int data)
{
    Dlist *ret = NULL;
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_head;
        while (p_temp != NULL)
        {
            if (p_temp->data == data)
            {
                if (ret == NULL)
                {
                    ret = dlist_new();
                }
                ret = dlist_append(ret, data);
            }
            p_temp = p_temp->p_next;
        }
    }
    return ret;
}
```

11. Le but est de créer une fonction dlist_reverse permettant "d'inverser" une liste chaînée. Tous les éléments doivent alors être inversés un par un (le premier doit se retrouver dernier, le

deuxième avant-dernier, ...). De plus, notre structure de liste nous donne une flexibilité incroyable car nous pouvons de choisir de partir soit en début de liste, soit en fin.

```
Dlist *dlist_reverse(Dlist *p_list)
{
    Dlist *ret = NULL;
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_tail;
        ret = dlist_new();
        while (p_temp != NULL)
        {
            ret = dlist_append(ret, p_temp->data);
            p_temp = p_temp->p_prev;
        }
    }
    return ret;
}
```